DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

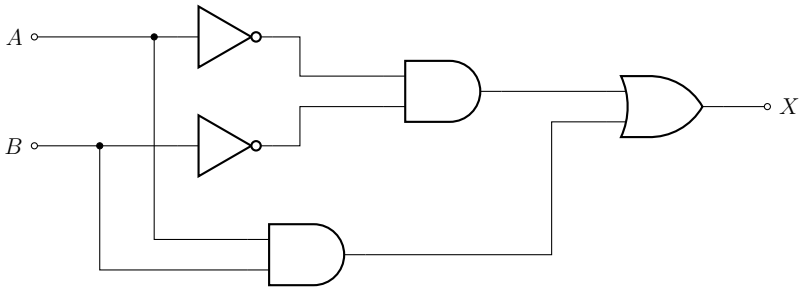
B.TECH. · Academic Year 2024-28

National Institute of Technology Bhopal

★

Analysis, Simplification and Arduino-Based Verification of XNOR Logic Circuit

GATE IN 2017 – Digital Logic Problem



DOCUMENTATION

Submitted by Shreya Agarwal

Subject Digital Electronics & Logic Design

Date June 9, 2026

CONTENTS

1. ABSTRACT

This report presents a systematic **analysis, simplification, and hardware verification** of a digital logic circuit drawn from the GATE Instrumentation Engineering (IN) 2017 examination paper. The circuit comprises two NOT gates, two AND gates, and one OR gate. Boolean algebra is applied to reduce the network, yielding the result:

$$\boxed{X = AB + \overline{A}\,\overline{B}}$$

This expression represents the **Exclusive-NOR (XNOR)** function, which evaluates to logic HIGH whenever both inputs are equal.
An **Arduino UNO** platform is subsequently employed to verify this result experimentally. An LED connected to pin D13 provides visual confirmation: the LED glows for input combinations $(0, 0)$ and $(1, 1)$, and extinguishes for $(0, 1)$ and $(1, 0)$, in complete agreement with the theoretical derivation.

2. OBJECTIVES

The following objectives guided the course of this project:

1. Analyse the structure of the given GATE IN 2017 logic circuit.
2. Derive the corresponding Boolean expression from the schematic.
3. Simplify the expression and identify the equivalent logic function.
4. Construct a complete truth table to verify the result.
5. Implement the logic on an Arduino UNO platform and observe output behaviour.
6. Write an equivalent AVR Assembly program and verify consistency.

3. PROBLEM STATEMENT

Determine the Boolean expression corresponding to the given GATE IN 2017 logic circuit (Fig. 3.16) and identify the relationship between inputs A, B and output X. Verify the result using a truth table, an Arduino C++ program, and an AVR Assembly implementation.

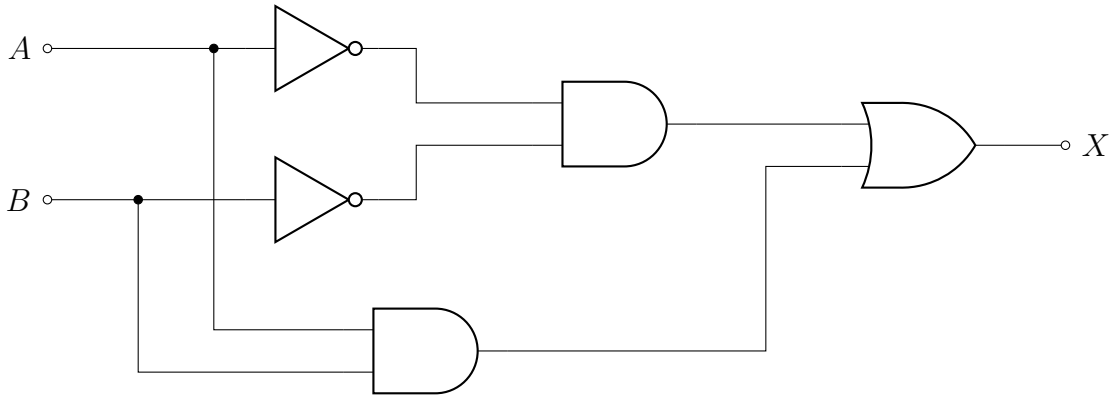


Figure 1: Given GATE IN 2017 Logic Circuit (Fig. 3.16)

The multiple-choice options given in the examination are:

a) $X = \overline{A}B + \overline{B}A$

b) $X = AB + \overline{B}A$

c) $X = AB + \overline{B}A$

d) $X = \overline{A}B + \overline{B}A$

4. CIRCUIT ANALYSIS AND SIMPLIFICATION

4.1 Step-by-Step Derivation

Step 1 — NOT Gate Outputs

The two inverters produce the complements of the inputs:

$$\overline{A} \quad \text{and} \quad \overline{B}$$

Step 2 — Upper AND Gate Output

The AND gate receiving both complements gives:

$$X_1 = \overline{A}\,\overline{B}$$

Step 3 — Lower AND Gate Output

The AND gate receiving the original inputs gives:

$$X_2 = AB$$

Step 4 — Final OR Gate Output

The OR gate combines X_1 and X_2:

$$X = X_1 + X_2 = \overline{A}\,\overline{B} + AB$$

$$\boxed{X = AB + \overline{A}\,\overline{B}}$$

The circuit implements an **XNOR** gate. The output is HIGH when both inputs are equal ($A = B$), and LOW when they differ ($A \neq B$).

5. TRUTH TABLE VERIFICATION

Table 1: Complete Truth Table — XNOR Function

A	B	X	Remark
0	0	**1**	$A = B$ ✓
0	1	0	$A \neq B$
1	0	0	$A \neq B$
1	1	**1**	$A = B$ ✓

The truth table confirms that $X = 1$ if and only if $A = B$, which is the defining property of the XNOR function.

6. HARDWARE IMPLEMENTATION

6.1 Components Required

Table 2: Bill of Materials

Component	Quantity
Arduino UNO Microcontroller Board	1
Light-Emitting Diode (LED)	1
220 Ω Current-Limiting Resistor	1
Breadboard	1
Jumper Wires	As required

6.2 Pin Assignment

Table 3: Arduino UNO Pin Assignment

Arduino Pin	Signal	Function
D11	Input A LED	LED indicator for input variable A
D12	Input B LED	LED indicator for input variable B
D13	Output X LED	LED indicator — HIGH when $A = B$ (XNOR)
5V	VCC	Power supply to LEDs and resistors
GND	GND	Common ground reference

6.3 Circuit Schematic

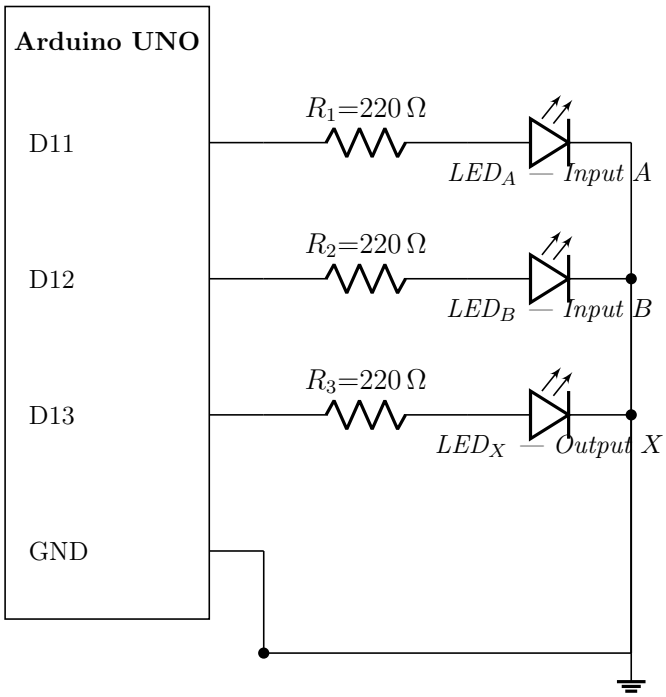


Figure 2: Arduino UNO Implementation Schematic

6.4 Working Principle

The Arduino firmware iterates through all four possible input combinations of A and B in a loop. LEDs on pins D11 and D12 display the current values of inputs A and B respectively, so the input state is always visible. The XNOR logic is computed in software and the result is driven on pin D13. The output LED glows whenever $X = 1$ (i.e. $A = B$) and extinguishes whenever $X = 0$ (i.e. $A \neq B$).

7. ARDUINO PROGRAM

```
const int A_LED = 11;    // Input A  -- LED indicator
const int B_LED = 12;    // Input B  -- LED indicator
const int Y_LED = 13;    // Output X -- LED indicator

void setup()
{
    pinMode(A_LED, OUTPUT);
    pinMode(B_LED, OUTPUT);
    pinMode(Y_LED, OUTPUT);
    Serial.begin(9600);
    Serial.println("A  B  X (XNOR)");
}

void loop()
{
    for (int i = 0; i < 4; i++)
    {
        int A = (i >> 1) & 1;  // Extract bit 1
        int B =  i       & 1;  // Extract bit 0

        // XNOR: output HIGH when A == B
        int X = (A == B) ? 1 : 0;

        digitalWrite(A_LED, A);   // Show current A on LED
        digitalWrite(B_LED, B);   // Show current B on LED
        digitalWrite(Y_LED, X);   // Show XNOR output on LED

        Serial.print(A); Serial.print("  ");
        Serial.print(B); Serial.print("  ");
        Serial.println(X);

        delay(1000);              // Hold for 1 second
    }
}
```

Listing 1: XNOR Verification — Arduino C++ Program

8. AVR ASSEMBLY PROGRAM

```
; Pin mapping:
;   PB3 = Arduino D11 -> A LED
;   PB4 = Arduino D12 -> B LED
;   PB5 = Arduino D13 -> X LED (XNOR output)

.include "m328pdef.inc"

.org 0x0000
    rjmp MAIN

MAIN:
    ldi  r16, 0b00111000    ; Set PB3, PB4, PB5 as outputs
    out  DDRB, r16

LOOP:
    ldi  r17, 0             ; Loop counter i = 0

NEXT:
    ; --- Extract A (bit1) and B (bit0) from counter ---
    mov  r18, r17
    lsr  r18
    andi r18, 0x01          ; r18 = A

    mov  r19, r17
    andi r19, 0x01          ; r19 = B

    ; --- Drive A LED on PB3 ---
    tst  r18
    breq A_LOW
    sbi  PORTB, 3           ; A = 1 -> LED ON
    rjmp A_DONE
A_LOW:
    cbi  PORTB, 3           ; A = 0 -> LED OFF
A_DONE:

    ; --- Drive B LED on PB4 ---
    tst  r19
    breq B_LOW
    sbi  PORTB, 4           ; B = 1 -> LED ON
    rjmp B_DONE
B_LOW:
    cbi  PORTB, 4           ; B = 0 -> LED OFF
B_DONE:

    ; --- Compute XNOR: X = 1 if A == B ---
    cp   r18, r19
    breq SET_HIGH
    cbi  PORTB, 5           ; A != B -> X LED OFF
    rjmp DONE
SET_HIGH:
    sbi  PORTB, 5           ; A == B -> X LED ON

DONE:
    rcall DELAY             ; ~1 second delay

```

```
    inc   r17
    cpi   r17, 4
    brne  NEXT              ; Repeat for all 4 combinations

    rjmp  LOOP              ; Restart cycle indefinitely

DELAY:
    ldi  r20, 100
L1: ldi  r21, 255
L2: ldi  r22, 255
L3: dec  r22
    brne L3
    dec  r21
    brne L2
    dec  r20
    brne L1
    ret
```

Listing 2: XNOR Verification — AVR Assembly (ATmega328P)

9. RESULTS

The Arduino UNO successfully iterated through all four input combinations of A and B. LEDs on D11 and D12 visually confirmed the current input state, while the output LED on D13 matched the XNOR truth table exactly:

- **D11 (A LED)** and **D12 (B LED)** toggled with each combination ✓
- **D13 (X LED) ON** for $(A, B) = (0, 0)$ and $(1, 1)$ ✓
- **D13 (X LED) OFF** for $(A, B) = (0, 1)$ and $(1, 0)$ ✓

Both the Arduino C++ firmware and the AVR Assembly implementation produced identical results across all three LEDs, demonstrating consistency at both levels of abstraction.

10. OPTIMISATION ANALYSIS

10.1 Gate Count Comparison

Table 4: Original Circuit vs. Direct XNOR Gate Implementation

Metric	Original Circuit	Simplified
NOT Gates	2	0
AND Gates	2	0
OR Gates	1	0
Total Gates	5	1 (XNOR IC)
Propagation Delay	3 stages	1 stage
Implementation	Discrete gates	74HCXX / CMOS

10.2 Advantages of Simplification

- **Reduced gate count** — five discrete gates replaced by a single XNOR gate (e.g. 74HC266), lowering component cost.
- **Lower propagation delay** — three cascaded gate delays reduced to one, improving switching speed.
- **Reduced power consumption** — fewer active logic elements draw less quiescent current.
- **Improved reliability** — fewer components result in fewer potential failure points and higher MTBF.
- **Smaller PCB footprint** — significant area savings in integrated circuit layouts and printed circuit board designs.

11. CONCLUSION

The given GATE IN 2017 logic circuit was systematically analysed using Boolean algebra. The step-by-step derivation established that:

$$X = AB + \overline{A}\,\overline{B}$$

This is the **XNOR** function — the output is logic HIGH when both inputs are equal, and logic LOW when they differ.

The truth table, Arduino C++ firmware, and AVR Assembly program all confirmed this result experimentally. This project underscores the practical value of Boolean simplification and gate-level optimisation in reducing hardware cost, minimising propagation delay, and streamlining design complexity in digital electronic systems.

— End of Report —